

LAST MILE

Guide d'accès, d'utilisation et de test de l'application logistique

Backend Node.js / Express / MySQL - Frontend React / Vite - Socket.io - PWA

Ce document résume comment démarrer l'application, comment y accéder, quels rôles utiliser, quelles fonctionnalités sont disponibles et quels endpoints API tester.

Important : les mots de passe existants ne sont pas lisibles car ils sont hashés avec bcrypt. Si un mot de passe est oublié, crée un nouveau compte via /register ou réinitialise-le en base.

1. Acces local

Backend API

```
cd D:\E-Commerce\last-mile-backend
npm run dev
```

- URL API : http://localhost:3000/api
- Socket temps reel : http://localhost:3000

Frontend React

```
cd D:\E-Commerce\last-mile-frontend
npm run dev
```

- URL application : http://127.0.0.1:5173/login
- URL alternative : http://localhost:5173/login

Mini-boutiques publiques

- http://127.0.0.1:5173/shop/merchant-3
- http://127.0.0.1:5173/shop/merchant-4
- http://127.0.0.1:5173/shop/styve-6
- http://127.0.0.1:5173/shop/daniel-7

Raccourci application

- Cliquez sur le bouton d'installation dans le header de l'application.
- Si le navigateur ne montre pas le prompt, ouvrez le menu du navigateur puis choisissez Installer l'application ou Ajouter à l'écran d'accueil.

2. Comptes locaux trouvés

Role	Email	Nom	Détails
Admin	styve@test.com	Styve Aristide	Accès global aux commandes, tracking et assignation.
Chauffeur	estherkentsa@gmail.com	Esther Kentsa	Peut recevoir des livraisons et envoyer la position GPS.
Chauffeur	driver.test.1782986988880@lastmile.local	Livreur Test	Compte chauffeur de test.
Marchand	styvekentsa@gmail.com	Edan Bradd	Boutique publique : merchant-4.
Marchand	merchant.dashboard@test.local	Marchand Dashboard	Boutique publique : merchant-3.
Marchand	styve@gmail.com	Styve	Boutique publique : styve-6.
Marchand	daniel@gmail.com	Daniel	Boutique publique : daniel-7.

3. Navigation et rôles

Marchand

- Connexion avec le rôle Merchant.
- Commandes : liste des commandes du marchand uniquement.
- Création : formulaire de création de commande.
- Mon catalogue : gestion des produits SocialHub.
- Tracking : suivi des colis du marchand.
- Profil : modification du nom, téléphone et avatar.

Admin

- Connexion avec le role Admin.
- Commandes : toutes les commandes visibles.
- Assignment : choix d un chauffeur connecte pour une commande en attente.
- Tracking : suivi global.
- Inspection profils : clic sur avatar depuis la console operations.
- Deblocage chauffeur : endpoint admin dedie.

Chauffeur

- Connexion avec le role Driver.
- Mes livraisons : liste des commandes assignees.
- Map : carte chauffeur avec vraie geolocalisation navigateur.
- Workflow statut : assigned -> picking -> in_transit -> delivered ou failed.

4. Donnees locales utiles

Type	Valeur	Observation
Commande	#12	Marchand 4, client Styve Kentsa, statut pending, paiement paid, MTN MoMo.
Commande	#11	Marchand 3, client Styve Kentsa, statut pending, paiement paid, MTN MoMo.
Produit	Air Force 1 Ultra	Produit merchant-3, stock actuel : 14.
Produit	Sac a Main Cuir Urbain	Produit merchant-4, stock actuel : 8.

5. Endpoints API principaux

Connexion

```
POST http://localhost:3000/api/auth/login
{
  "email": "email@test.com",
  "password": "motdepasse"
}
```

Inscription

```
POST http://localhost:3000/api/auth/register
{
  "name": "Nom Test",
  "email": "test@example.com",
  "password": "123456",
  "phone": "+237600000000",
  "role": "merchant"
}
```

Creation de commande

```
POST http://localhost:3000/api/orders
Authorization: Bearer <token_marchand>
{
  "client_name": "Client Test",
  "client_phone": "+237690000000",
  "delivery_address": "Bonamoussadi",
  "latitude": 4.0908,
  "longitude": 9.7468,
  "description": "Petit colis fragile"
}
```

Liste des commandes

```
GET http://localhost:3000/api/orders
Authorization: Bearer <token>
```

- Admin : toutes les commandes payees ou manuelles.
- Marchand : uniquement ses commandes.
- Chauffeur : uniquement les commandes qui lui sont assignees.

Assignment chauffeur

```
PUT http://localhost:3000/api/orders/12/assign
Authorization: Bearer <token_admin>
{
  "driver_id": 5
}
```

Mise a jour statut

```
PUT http://localhost:3000/api/orders/12/status
Authorization: Bearer <token_admin_ou_chauffeur>
{
  "status": "picking"
}
```

Historique tracking

```
GET http://localhost:3000/api/orders/12/tracking
Authorization: Bearer <token>
```

Position GPS chauffeur

```
PUT http://localhost:3000/api/drivers/location
Authorization: Bearer <token_chauffeur>
{
  "latitude": 4.0652,
  "longitude": 9.7503,
  "accuracy": 20,
  "captured_at": "2026-07-31T12:00:00.000Z",
  "heading": 90,
  "speed": 1.2
}
```

6. SocialHub Checkout

Catalogue public

```
GET http://localhost:3000/api/public/shops/merchant-4
```

Checkout public

```
POST http://localhost:3000/api/public/checkout
{
  "shop_slug": "merchant-4",
  "cart": [
    { "product_id": 7, "quantity": 1 }
  ],
  "customer": {
    "name": "Client WhatsApp",
    "phone": "+237677000000",
    "email": "client@test.com",
    "district": "Akwa"
  },
  "payment_method": "mtn_momo"
}
```

Webhook paiement

```
POST http://localhost:3000/api/payments/webhook
Header: flutterwave-signature: <signature_fournisseur>
```

En production, la commande devient visible comme commande payee uniquement apres confirmation du webhook Flutterwave.

7. Fonctionnalités actives

- Authentification JWT avec roles.
- Persistance de session avec Remember me.
- Dashboard adaptatif desktop/mobile.
- Theme clair/sombre.
- Traduction EN/FR.
- Commandes filtrées par role.
- Recherche instantanée dans les commandes et le tracking.
- Rafraîchissement automatique sans bouton refresh obligatoire.
- Socket.io pour commandes, statuts, paiements et GPS.
- Tracking Leaflet sombre.
- Vraie geolocalisation chauffeur via navigator.geolocation.watchPosition.
- Historique chronologique de livraison.
- Anti-intrusion chauffeur avec blocage après accès interdits répétés.
- Déblocage chauffeur par admin.
- Profil utilisateur avec avatar.
- Lightbox avatar et inspection profil admin.
- Catalogue marchand avec images, stock, création, suppression.
- Boutique publique /shop/:slug.
- Checkout Mobile Money / carte via structure Flutterwave.
- Installation PWA en raccourci appareil.

8. Procédure de test rapide

Etape	Action	Résultat attendu
1	Demarrer backend puis frontend.	API disponible sur 3000, interface sur 5173.
2	Créer un compte marchand via /register.	Redirection vers /login après création.
3	Se connecter comme marchand.	Accès aux onglets Commandes, Création, Catalogue, Tracking, Profil.
4	Publier un produit dans Mon catalogue.	Le produit apparaît aussi dans /shop/.
5	Commander depuis la boutique publique.	Création d'une commande pending avec paiement pending ou paid selon confirmation.
6	Se connecter comme admin.	Voir les commandes et assigner un chauffeur connecté.
7	Se connecter comme chauffeur sur téléphone ou navigateur avec GPS autorisé.	La position exacte remonte au tracking.
8	Changer les statuts chauffeur.	Les dashboards se mettent à jour via Socket.io.

9. Variables locales importantes

Configuration locale

```
Backend .env local attendu
PORT=3000
NODE_ENV=development
CORS_ORIGIN=http://localhost:5173,http://127.0.0.1:5173
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=e-commerce
JWT_SECRET=<secret_local>
GPS_MAX_ACCURACY_METERS=100
FRONTEND_URL=http://127.0.0.1:5173
BACKEND_URL=http://localhost:3000

Frontend .env local attendu
VITE_API_URL=http://localhost:3000
VITE_SOCKET_URL=http://localhost:3000
VITE_GPS_MAX_ACCURACY_METERS=100
```

10. Notes importantes

- Les mots de passe ne peuvent pas être récupérés en clair depuis la base.
- Les chauffeurs disponibles pour assignation sont ceux connectés au Socket en temps réel.
- Le tracking exact nécessite un appareil avec GPS autorisé et précision inférieure ou égale à 100 mètres.
- Sur iPhone/Safari, le prompt PWA ne peut pas toujours être déclenché par code : il faut passer par le menu de partage.
- En local, Flutterwave réel peut nécessiter des clés de test et une URL webhook publique via tunnel type ngrok.